

ITEC 4366 Advanced Web Development

Lab 3: JSON Parsing in PHP

Purpose	Use PHP to read, decode, filter, and display JSON data in a small server-side web application.
Estimated Effort	3 to 5 hours, depending on comfort with PHP loops, arrays/objects, and query string values.
Main Skills	JSON structure, file_get_contents(), json_decode(), foreach loops, reusable PHP includes, query string filtering, responsive data display.
Required Folder Name	LastNameLab3, such as StinesLab3, inside your local web server htdocs folder.
Submission Format	A single .zip file containing your full Lab 3 folder, source files, JSON file, CSS, images if used, and required screenshots.
Point Value	25 points

Background

Modern web applications often exchange data using JSON, especially when working with APIs. In this lab, we will keep the environment simple by using a local JSON file instead of requiring students to sign up for external cloud services or API keys.

You will use PHP on your local web server to read a JSON file, decode it into a usable PHP data structure, and display that data inside a styled web page. This gives you practice with the same general pattern used when a server-side application consumes API data: retrieve the data, parse it, loop through it, filter it, and present it clearly to the user.

This lab also reinforces the difference between client-side and server-side development. HTML, CSS, and JavaScript run in the browser. PHP runs on the server before the completed page is sent to the browser.

Learning Outcomes

- Explain why JSON is commonly used to exchange structured data in web applications and APIs.
- Use PHP to load and decode a JSON file into a usable data structure.
- Use a foreach loop to display JSON data dynamically in HTML.
- Create reusable PHP template components using include or require statements.
- Use query string values with \$_GET or \$_REQUEST to filter displayed data.
- Style dynamically generated content so that tables, navigation, headers, and footers are readable and responsive.
- Validate and test the final output as viewed in the browser.

Required Starting Point

- Use the provided starter ZIP file for Lab 3. The JSON file is the most important required file. Starter code may be used as a guide, but your final solution should reflect your own implementation and design decisions.
- Your local server environment should already be working from Labs 1 and 2. XAMPP is recommended for consistency with course examples, but another local PHP environment is acceptable if you can demonstrate that your work runs correctly.
- Place your working folder inside the htdocs folder and name it using the required format: LastNameLab3.

Task 1: Set Up the Lab 3 Website

- Download and extract the provided Lab 3 starter ZIP file.
- Rename the extracted folder using the format LastNameLab3.
- Move the folder into your local web server htdocs directory.
- Confirm that the site loads at <http://localhost/LastNameLab3> or <http://127.0.0.1/LastNameLab3>.

- Take a screenshot showing the browser URL and the Lab 3 site running on localhost. Save this screenshot inside your Lab 3 folder.

Task 2: Read and Decode the JSON File

- Use PHP `file_get_contents()` to read the JSON file into a variable.
- Use `json_decode()` to convert the JSON text into a PHP data structure.
- Use `print_r()` or `var_dump()` temporarily while testing so you can inspect the structure of the decoded data.
- Remove or comment out raw debugging output before submitting the final version.
- Add short code comments that explain the main purpose of each step. Do not over-comment every line like a nervous robot trying to pass inspection.

Task 3: Display JSON Data on the Home Page

- Use a `foreach` loop to go through the decoded JSON data.
- Display each item in a meaningful HTML structure, such as cards, rows, or a table.
- Include at least six fields from the JSON dataset in your output. The original element name plus at least five additional properties is acceptable.
- Use readable labels so users understand what each field represents.
- Use `htmlspecialchars()` or another appropriate escaping technique when printing data into HTML. This helps reinforce safe output habits when displaying dynamic content.
- The page should look like a designed data-driven website, not just a raw dump of JSON data.

Task 4: Reuse Template Components

- Split shared layout code into reusable PHP component files. You may use names such as `_top.php` and `_bottom.php`, or `_header.php` and `_footer.php`.
- The top component should contain shared opening HTML, metadata, stylesheet links, header content, and navigation.
- The bottom component should contain shared footer content and closing HTML.
- Your main pages should include the shared components with `include` or `require` statements.
- Use a page-specific variable, such as `$title`, so each page can set its own browser title before including the shared top component.
- Keep the component system organized. Reuse is good. Spaghetti code is still spaghetti, even if served with PHP sauce.

Task 5: Create a Filtered Type Page

- Create a `type.php` page that displays a filtered list of records from the JSON dataset.
- Modify your navigation links so they pass a query string value to `type.php`. For example, `type.php?phase=gas` or `type.php?category=metal` depending on the dataset structure.
- Use `$_GET` or `$_REQUEST` to retrieve the query string value.
- Filter the decoded JSON data so only matching records appear on the page.
- Display the filtered results in a styled table or responsive card layout.
- Include at least five properties from the dataset in the filtered output.
- Handle missing or unexpected query string values gracefully. For example, show a helpful message instead of throwing warnings all over the page like PHP confetti.

Task 6: Create a Families or Categories Page

- Create a `families.php` page, `categories.php` page, or similarly named page based on the structure of the JSON file.
- Loop through the JSON data and collect the family, category, type, or similar grouping value from each item.
- Use `array_push()` or a similar approach to build an array of those grouping values.
- Use `array_unique()` to identify the unique group values.

- Display the unique values in a clean list, table, or set of links.
- When possible, make each family or category link to the filtered type page so users can explore the dataset.

Task 7: Improve Design and Usability

- Use Bootstrap 5, another current CSS framework, or well-organized custom CSS.
- Apply a consistent color scheme across the site.
- Style the navigation so users can clearly move between pages.
- Create distinct header and footer styles.
- Style tables or cards so the data is readable on both desktop and mobile screens.
- Use spacing, headings, labels, and grouping to make the data easier to scan.
- Avoid relying only on color to communicate meaning. Use labels, headings, or icons where appropriate.

Optional Challenge Features

- Create a details.php page. Each item on the home page should include a Read More link that passes a unique value through the query string, such as details.php?element=H.
- Use JavaScript to add searching, sorting, or filtering on the client side.
- Add a “no results found” message when a filter does not match any records.
- Add active navigation styling so the current page or selected filter is visually identified.
- Use small helper functions in PHP to reduce repeated code.

Validation and Testing Requirements

- View the generated page source in the browser and validate the final HTML output using the W3C Markup Validation Service.
- Fix major HTML errors before submission. Dynamic PHP code does not excuse broken final HTML output.
- Test the site using multiple query string values.
- Test what happens when the query string value is missing or invalid.
- Confirm that all navigation links work.
- Confirm that the required JSON data displays correctly on localhost.
- Take at least one screenshot showing your completed site running on localhost with the browser URL visible.

AI Use and Developer Responsibility

You may use AI tools to brainstorm, troubleshoot, explain code, generate starter ideas, or assist with debugging. However, you are ultimately responsible for the accuracy, functionality, security, originality, and completeness of the work you submit.

Submitting AI-generated code without understanding, testing, adapting, and documenting it is not acceptable. You should be able to explain your PHP includes, JSON parsing process, loops, filtering logic, query string handling, and design choices. AI is allowed as a development aid, but the submitted work must reflect your own learning, judgment, and responsibility as the developer.

Submission Checklist

- Your complete LastNameLab3 folder is included in the ZIP file.
- The JSON file is included and used by the PHP pages.
- The site runs correctly from localhost.
- The index page reads and displays JSON data dynamically using PHP.
- Reusable PHP template components are used across pages.
- A filtered page uses query string data to display different results.
- A family, category, or grouping page displays unique values from the dataset.
- CSS or a framework is used to create a readable responsive design.
- Required screenshots are included in the folder.
- The final submission is uploaded to the D2L Assignments Dropbox as a .zip file.

